

CS-141

lecture06

Strings

String methods

- What does a String look like?
 - char primitive type
- String methods
 - length()
 - charAt(int index)
 - substring(int start)
 - substring(int start, int end)
- for loops plus strings

What are strings made up of?

What are Strings made up of?

characters!

What is a character?

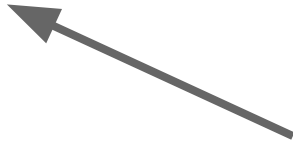
A single character in Java is a primitive type,
called `char`

Literal chars use single quotes

```
char ch='a';
```

Note that this won't compile (because double-quotes means a String):

```
char ch="a";
```



Don't use double-quotes for chars! This will not compile

The String "hello" actually looks like this:

h	e	l	l	o
0	1	2	3	4

Each character is numbered, starting at 0

Why don't we start counting at 1?

Because we're in bizzaro world, where 8/10 is 0!

Strings are special

Strings are reference types.

What other reference types have we seen?

Strings are special

Strings are reference types.

What other reference types have we seen?

Scanner

String

IDrawGrid

Color (**believe it or not**)

What do reference types have in common?

What do reference types have in common?

Several answers:

- must be compared with `.equals()` instead of `==`
- have methods
- are references to data of a variable and potentially large size
- are passed into methods by reference
 - rather than by value as we do with primitives

Reference types have methods

What methods does Scanner have?

Reference types have methods

What methods does Scanner have?

```
String next()
```

```
String nextLine()
```

```
int nextInt()
```

```
double nextDouble()
```

What methods does IDrawGrid have?

```
void setColor(int row, int col, Color color)
int getNumRows()
int getNumCols()
```

Let's skip over Color

It has methods, such as the ability to get the red, green, and blue values of the color, but we don't care about those for now

What methods does String have?

We don't know yet! But it makes sense that a String would have methods, since the other reference types all do.

String methods

```
int length()
```

```
String substring(int start)
```

```
String substring(int start, int end)
```

```
int indexOf(String str)
```

```
char charAt(int index)
```

String methods

`int length()` : Returns the length of a String

`String substring(int start)` : Returns a new String of only the characters at start or later.

`String substring(int start, int end)` : returns a new String of the chars between start (inclusive) and end (exclusive)

`int indexOf(String str)` : return the index where the given String first occurs

`char charAt(int index)` : returns the character at the given index

instance methods

These methods are all *instance methods*.

They all require an *instance* to do their work.

You want the length of a string? Which instance of string?

You want a substring? A substring of which string instance?

Invoke methods on a particular instance

```
String s1="Knox College";  
String s2="Galesburg";  
int len=s1.length();
```

We are asking for the length of a *particular instance* of String.

What does this print?

```
String str="The Knox Student";  
System.out.println(str.length());
```

A: 14

B: 15

C: 16

D: 3

E: unknown

What does this print?

```
String str="The Knox Student";  
System.out.println(str.length());
```

C: 16

16 characters long! Spaces count in the length of a String.

What does this print?

```
String s="The Knox Student";  
System.out.println(s.substring(4));
```

- A: Student
- B: nox Student
- C: Knox Student
- D: Knox Student (i.e. with a space in front)
- E: Knox

What does this print?

```
String s="The Knox Student";  
System.out.println(s.substring(4));
```

A: Student

B: nox Student

C: Knox Student

D: Knox Student (i.e. with a space in front)

E: Knox

What is the type and return value of this expression?

```
String s="Knox College";  
s.charAt(3);
```

A: 'o', String

B: 'o', char

C: 'x', String

D: 'x', char

E: 3, int

What is the type and return value of this expression?

```
String s="Knox College";  
s.charAt(3);
```

A: 'o', String

B: 'o', char

C: 'x', String

D: 'x', char

E: 3, int

What does this print?

```
String s1="Knox College";  
int i=s1.indexOf(" ");  
String s2=s1.substring(i+1);  
System.out.println(s2);
```

- A: Knox
- B: Knox_ (i.e. with a space after it)
- C: _College (i.e. with a space before it)
- D: College
- E: Knox College

What does this print?

```
String s1="Knox College";  
int i=s1.indexOf(" ");  
String s2=s1.substring(i+1);  
System.out.println(s2);
```

D: College

The call to `indexOf()` gives us the location of the first space. Then the `substring` gives us everything after that space.

What does this print?

```
String s="Knox College";  
s.substring(5);  
System.out.println(s);
```

A: Knox

B: Knox_

C: _College

D: College

E: Knox College

What does this print?

```
String s="Knox College";  
s.substring(5);  
System.out.println(s);
```

E: Knox College

The substring() method does not change the String! It returns a new String.

What does this print?

```
String s="Knox College";  
s = s.substring(5);  
System.out.println(s);
```

This will print "College". Notice the reassignment of s to the result of the substring operation.

Count the number of occurrences of 'a' in a given String

What language feature do we combine with String to count things?

Count the number of occurrences of 'a' in a given String

What language feature do we combine with String to count things?

for loops!

Count the number of occurrences of 'a' in a given String

```
String s = scan.next();
int count = 0;
for (int i=0; i<s.length(); i++) {
    if (s.charAt(i) == 'a') {
        count++;
    }
}
System.out.println(count);
```

Write a static method...

Called countA that takes a String as a parameter, and returns the number of times the letter a occurs in the given String.

countA method

step 1, get the function header

```
public static int countA(String s)
{
    // body goes here
}
```

countA method

step 1, get the function header

```
public static int countA(String s) {  
    int count=0;  
    for (int i=0; i<s.length(); i++) {  
        if (s.charAt(i)=='a') {  
            count++;  
        }  
    }  
    return count;  
}
```

For what inputs will this print true?

```
String s1 = scanner.next();  
s1.toLowerCase();  
System.out.println(s1.equals("elves"));
```

- A. 1
- B. 1,2
- C. 1,2,3
- D. 1,2,3,4
- E. none of the above

Inputs	
1	"elves"
2	"Elves"
3	"ELVES"
4	"eLvEs"

For what inputs will this function return true?

```
String s1 = scanner.next();
```

```
s1.toLowerCase();
```

```
System.out.println(s1.equals("elves"));
```

This doesn't actually change s1!

A. 1 It should be:

B. 1,2

C. 1,2,3 `s1=s1.toLowerCase();`

D. 1,2,3,4

E. none of the above

Only #1 works as currently written!

Inputs	
1	"elves"
2	"Elves"
3	"ELVES"
4	"eLvEs"